

Experiment 8 — Random number generation

Mathilda — execution-speed experiments

Contents

Experiment 8 — Random number generation	1
Hypothesis	1
What was wrong	1
The alternatives, measured	1
Three fixes beyond the generator	2
Results	2
What this cost	2
Distribution, checked rather than assumed	3
Still open	3
Why Mathilda is not the fastest here, and what it would take	3

Experiment 8 — Random number generation

Date: 2026-07-30 · Commit: 121d264 · Code: `src/random.c`, `src/random.h` · Result: 53×–70×

Common method in [README.md](#).

Hypothesis

The Monte Carlo π benchmark ran 2.73 s against Mathematica's 286 ms — 9.5× behind, and the arithmetic in it is a comparison and an addition. Either the vectorised path was not being taken, or the generator itself was the cost. A profile said the second: `random_uniform_01` had more samples than everything else in the benchmark combined.

What was wrong

Every machine-precision draw went through GMP. To produce 53 bits, `random_uniform_01` did two `mpz_inits`, recomputed 2^{53} with `mpz_ui_pow_ui`, called `mpz_urandomm` — a bignum modular draw — and cleared both. Two heap allocations and an arbitrary-precision modulo per double.

That is not a subtle inefficiency. It is using the arbitrary-precision path for a value that fits in a register, and it had been there since `RandomReal` was written, because the generator was built for `RandomInteger` over unbounded ranges and `RandomReal` was layered on top of it.

The alternatives, measured

Not assumed — four candidates, 10^7 draws each:

generator	ns/draw
mpz per call (what shipped)	117.2
hoisted mpz scratch (keep GMP, stop allocating)	18.3
gmp_urandomb_ui	9.0
xoshiro256++	2.4

Hoisting the scratch — the obvious minimal fix — recovers 6.4× of the available 49×. The generator, not the allocation, was the floor.

xoshiro256++ (256-bit state, period $2^{256}-1$, passes BigCrush) now serves the machine-precision paths. GMP keeps the bignum integer ranges and the MPFR draws, so arbitrary-precision behaviour is unchanged. Both generators are seeded and saved/restored together, so `SeedRandom` stays one coherent operation and `random_push_seed/random_pop_seed` still leave the user's stream untouched.

Three fixes beyond the generator

Swapping the generator alone left `RandomInteger` at 3× `RandomReal`, which made no sense for the cheaper operation. Three separate causes:

1. Build the result from an `int64` instead of allocating an mpz-backed `Expr` and normalising it back down.
2. Fill a packed `int64` buffer directly, as `RandomReal` already did, rather than building one `Expr` per element and offering the list to `pack_offer`.
3. Lemire's nearly-divisionless bounded draw. The textbook rejection form costs two 64-bit divisions per element and was alone responsible for a 3× gap.

Results

Benchmark	Mathilda before	Mathilda after	Mathematica 14.0	Python
<code>RandomReal[{0,1}, 10⁷]</code>	1.22 s	22.9 ms	—	67.4 ms
<code>RandomInteger[{0,100}, 10⁷]</code>	1.73 s	24.8 ms	—	69.3 ms
<code>RandomInteger[{0,255}, 10⁷]</code>	—	19.7 ms	—	—
Monte Carlo π , 10 ⁷ samples	2.73 s	203 ms	286 ms	205 ms

NumPy's bulk draws are `np.random.rand / np.random.randint` at 10⁷, measured here; Mathilda is ~2.9× faster on both, which says the generator is no longer the constraint. The bulk-draw rows are Mathilda against NumPy only: Mathematica's `RandomReal` draws from a different generator with different guarantees, so a bare draw-rate row would compare two unlike things. The Monte Carlo row is the like-for-like one — same algorithm, same sample count, checked answer — and Mathilda is 1.41× faster than Mathematica and level with NumPy. (In the full three-system run of 2026-07-31 the same row reads 162 ms / 250 ms / 205 ms, so Mathilda is now 1.54× Mathematica and 1.26× NumPy on it.)

The power-of-two row is worth its line: `{0,255}` needs no rejection at all, just a mask, and it is the fastest of the three.

What this cost

The seeded stream changed. That is unavoidable when the generator changes, but it had a consequence worth recording:

PossibleZeroQ's Schwartz–Zippel sampler drew its points by evaluating a `RandomInteger` call, so it inherited whatever generator the builtin used. Those points are part of a decision procedure — seeded from the expression's structural hash so that a verdict is a pure function of its input — and moving the generator moved the

integrator's answers with it. After the change, `Integrate[E^(Log[x]^2), x]`, which is non-elementary and must stay unintegrated, was claimed as solved.

The fix was to give the sampler its own generator rather than to revert. The lesson is general: a subsystem that samples for a decision must not share the user-facing stream, or its answers become a function of unrelated changes.

Twelve concrete outputs in [docs/spec/builtins/random-number-generation.md](#) were regenerated. No test pinned a specific value — the reproducibility tests assert only that a seed reproduces its own sequence, which is the right assertion and is why nothing else broke.

Distribution, checked rather than assumed

A faster generator that is not uniform is not a faster generator.

- Decile counts within 1% over 10^6 draws.
- Range endpoints respected and hit.
- Lag-1 autocorrelation below 0.01.
- Six-way `RandomInteger[{1, 6}]` uniform to 0.4%.
- `RandomChoice` balanced to 2%.
- Bounded draws are exactly uniform: the power-of-two path is a mask and the general path rejects, so there is no modulo bias.

Still open

Nothing specific to this experiment. `RandomReal` at 2.3 ns/draw is below the cost of storing the result, so the next constraint is memory bandwidth, not the generator.

Why Mathilda is not the fastest here, and what it would take

This is the one experiment in the suite where Mathilda is fastest on every row it can be compared on:

row	Mathilda	best other	
<code>RandomReal[{0, 1}, 10⁷]</code>	22.9 ms	67.4 ms (NumPy)	2.94× ahead
<code>RandomInteger[{0, 100}, 10⁷]</code>	24.8 ms	69.3 ms (NumPy)	2.79× ahead
Monte Carlo π , 10 ⁷ samples	203 ms	205 ms (NumPy)	level, and 1.41× ahead of Mathematica

`xoshiro256++` at 2.4 ns/draw is close to the arithmetic floor for a 64-bit-state generator, so there is little left to take.

If it had to be faster

1. A vectorised generator. `xoshiro256++` has four independent 64-bit lanes and vectorises to AVX2 almost mechanically, which is roughly what NumPy's PCG64DXSM-based Generator does. Expected: 2.4 → ~0.8 ns/draw.
2. Fuse the draw with its consumer. The Monte Carlo row draws 2×10^7 values into two arrays and then reads them once. A generator that fills a block and hands it straight to the consuming expression — the same fusion as plan 9.2 — removes 160 MB of traffic.

Neither is worth doing before the items in experiments 7, 14 and 17, which are between 2× and 100×. This row is recorded as done.